

```
minitinerary = itinerary
minidx = 0

maxitin = len(itinerary) * 50000
for t in range(0,maxitin):
    newitinerary = perturb_sa3(cities,newitinerary,t,maxitin)

return(newitinerary.copy())
```

[Listing 6-7](#): This function performs the full simulated annealing process and returns an optimized itinerary.

Next, we call our `siman()` function and compare its results to the results of our nearest neighbor algorithm:

```
np.random.seed(random_seed)
itinerary = list(range(N))
nnitin = donn(cities,N)
nnresult = howfar(genlines(cities,nnitin))
simanitinerary = siman(itinerary,cities)
simanresult = howfar(genlines(cities,simanitinerary))
print(nnresult)
print(simanresult)
print(simanresult/nnresult)
```

When we run this code, we find that our final simulated annealing function yields an itinerary with distance 5.32. Compared to the nearest-neighbor itinerary distance of 6.29, this is an improvement of more than 15 percent. This may seem underwhelming to you: we spent more than a dozen pages grappling with difficult concepts only to shave about 15 percent from our total distance. This is a reasonable complaint, and it may be that you never need to have better performance than the performance offered by the nearest neighbor algorithm. But imagine offering the CEO of a global logistics company like UPS or DHL a way to decrease travel costs by 15 percent, and seeing the pupils of their eyes turn to dollar signs as they think of the billions of dollars this would represent. Logistics remains a major driver of high costs and environmental pollution in every business in the world, and doing well at solving the TSP will always make a big practical difference. Besides this, the TSP is extremely important academically, as a benchmark for comparing optimization methods and as a gateway to investigating advanced theoretical ideas.